

Uniapp 快速开发自定义模板

一、模板概述

本模板基于 Uniapp 3.x 构建，适配多端（微信小程序、App、H5 等），整合了项目开发中高频使用的基础配置、功能模块、工具类及开发规范。核心优势：

- **多端适配**：内置多端样式兼容方案，解决小程序与 App 样式差异问题
- **架构清晰**：采用模块化目录结构，区分页面、组件、工具等核心模块
- **功能预置**：封装请求、存储、权限等基础功能，减少重复开发
- **规范统一**：集成 ESLint 代码规范，统一命名与编码风格

适用场景：中小型 Uniapp 项目（电商、资讯、工具类 App/小程序等），可根据实际需求灵活扩展。

二、目录结构

Code block

```
1  uniapp-quick-template/
2  |—— api/                // 接口请求模块
3  |   |—— index.js        // 请求拦截配置
4  |   |—— user.js         // 用户相关接口
5  |   |—— common.js       // 通用接口
6  |—— components/         // 通用组件
7  |   |—— base/           // 基础组件（按钮、输入框等）
8  |   |   |—— CuButton.vue // 自定义按钮
9  |   |   |—— CuInput.vue  // 自定义输入框
10 |   |—— layout/         // 布局组件
11 |   |   |—— NavBar.vue   // 导航栏组件
12 |   |—— business/       // 业务组件（根据需求扩展）
13 |—— pages/              // 页面模块
14 |   |—— index/          // 首页
15 |   |—— login/          // 登录页
16 |   |—— mine/           // 我的页面
17 |—— static/             // 静态资源
18 |   |—— img/            // 图片资源（按模块分类）
19 |   |—— style/          // 全局样式
20 |       |—— base.scss    // 基础样式
21 |       |—— theme.scss   // 主题样式
22 |—— store/              // 状态管理（Pinia）
23 |   |—— index.js        // Pinia 实例配置
24 |   |—— modules/        // 模块划分
```

```
25 |       └─ userStore.js // 用户状态
26 | └─ utils/              // 工具类
27 |   └─ request.js       // 请求工具（二次封装）
28 |   └─ storage.js       // 存储工具（兼容多端）
29 |   └─ auth.js          // 权限工具
30 |   └─ common.js        // 通用工具
31 | └─ pages.json          // 页面路由配置（预置常用配置）
32 | └─ manifest.json      // 应用配置（多端权限预置）
33 | └─ App.vue            // 应用入口
34 | └─ main.js            // 入口文件（初始化配置）
35 | └─ eslintrc.js        // ESLint 配置
```

三、核心配置文件模板

1. main.js（入口初始化）

Code block

```
1  import { createSSRApp } from 'vue'
2  import * as Pinia from 'pinia'
3  import App from './App.vue'
4  // 引入全局样式
5  import './static/style/base.scss'
6  // 引入通用组件（全局注册）
7  import CuButton from './components/base/CuButton.vue'
8  import NavBar from './components/layout/NavBar.vue'
9
10 export function createApp() {
11   const app = createSSRApp(App)
12   // 注册 Pinia 状态管理
13   app.use(Pinia.createPinia())
14
15   // 全局注册通用组件
16   app.component('CuButton', CuButton)
17   app.component('NavBar', NavBar)
18
19   // 全局挂载工具类（可选）
20   import('./utils/common.js').then(utils => {
21     app.config.globalProperties.$utils = utils
22   })
23
24   return {
25     app,
26     Pinia
27   }
```

2. pages.json（路由与窗口配置）

Code block

```
1  {
2    "pages": [
3      // 首页（必须放在第一位）
4      {
5        "path": "pages/index/index",
6        "style": {
7          "navigationBarTitleText": "首页",
8          "navigationStyle": "custom", // 自定义导航栏
9          "enablePullDownRefresh": true // 开启下拉刷新
10       }
11     },
12     // 登录页
13     {
14       "path": "pages/login/index",
15       "style": {
16         "navigationBarTitleText": "登录",
17         "navigationStyle": "custom",
18         "disableScroll": true // 禁止页面滚动
19       }
20     },
21     // 我的页面
22     {
23       "path": "pages/mine/index",
24       "style": {
25         "navigationBarTitleText": "我的",
26         "navigationStyle": "custom"
27       }
28     }
29   ],
30   // 全局窗口配置
31   "globalStyle": {
32     "navigationBarBackgroundColor": "#ffffff",
33     "navigationBarTextStyle": "black",
34     "navigationBarTitleText": "Uniapp 模板",
35     "backgroundColor": "#f5f5f5",
36     // 适配小程序胶囊位置
37     "paddingTop": "safe-area",
38     "paddingBottom": "safe-area"
39   },
40   // 底部 Tab 配置（如需）
```

```

41     "tabBar": {
42       "color": "#666666",
43       "selectedColor": "#409eff",
44       "backgroundColor": "#ffffff",
45       "borderStyle": "black",
46       "list": [
47         {
48           "pagePath": "pages/index/index",
49           "text": "首页",
50           "iconPath": "static/img/tab/home.png",
51           "selectedIconPath": "static/img/tab/home-active.png"
52         },
53         {
54           "pagePath": "pages/mine/index",
55           "text": "我的",
56           "iconPath": "static/img/tab/mine.png",
57           "selectedIconPath": "static/img/tab/mine-active.png"
58         }
59       ]
60     },
61     // 分包配置 (大型项目用)
62     "subPackages": [],
63     // 预加载配置
64     "preloadRule": {
65       "pages/index/index": {
66         "network": "all",
67         "packages": ["__APP__"]
68       }
69     }
70   }

```

3. manifest.json (多端基础配置)

Code block

```

1   {
2     "name": "uniapp-quick-template",
3     "appid": "__UNI__XXXXXX", // 替换为自己的 AppID
4     "description": "Uniapp 快速开发模板",
5     "versionName": "1.0.0",
6     "versionCode": 100,
7     "locale": "zh-CN",
8     "app-plus": {
9       "usingComponents": true,
10      "navigationBar": {
11        "backgroundColor": "#ffffff"

```

```
12     },
13     // App 权限配置
14     "permissions": {
15         "android.permission.INTERNET": {
16             "request": "always",
17             "prompt": "应用需要网络权限以获取数据"
18         },
19         "android.permission.READ_PHONE_STATE": {
20             "request": "whenNeed",
21             "prompt": "应用需要获取设备信息以完成登录"
22         }
23     },
24     // 启动页配置
25     "splashscreen": {
26         "alwaysShowBeforeRender": true,
27         "waiting": true,
28         "autoclose": true,
29         "delay": 500
30     }
31 },
32 // 微信小程序配置
33 "mp-weixin": {
34     "appid": "wxXXXXXX", // 替换为自己的小程序 AppID
35     "setting": {
36         "urlCheck": false,
37         "es6": true,
38         "postcss": true
39     },
40     "usingComponents": true,
41     "permission": {
42         "scope.userLocation": {
43             "desc": "你的位置信息将用于定位功能"
44         }
45     }
46 },
47 // H5 配置
48 "h5": {
49     "title": "Uniapp 模板",
50     "domain": "",
51     "router": {
52         "mode": "hash"
53     },
54     "devServer": {
55         "port": 8080,
56         "proxy": {
57             "/api": {
58                 "target": "https://api.example.com",
```

```
59         "changeOrigin": true,
60         "pathRewrite": {
61             "^/api": ""
62         }
63     }
64 }
65 }
66 }
67 }
```

四、核心功能模块模板

1. 网络请求工具 (utils/request.js)

Code block

```
1  import { getToken, removeToken } from './auth'
2  import { showToast } from './common'
3
4  // 基础配置
5  const BASE_URL = process.env.NODE_ENV === 'development' ? '/api' :
  'https://api.example.com'
6  const TIMEOUT = 10000
7
8  // 请求拦截器配置
9  const requestInterceptor = (options) => {
10    // 添加请求头
11    options.header = {
12      ...options.header,
13      'Content-Type': options.contentType || 'application/json',
14      'Authorization': `Bearer ${getToken() || ''}` // 携带 Token
15    }
16    // 拼接请求地址
17    options.url = BASE_URL + options.url
18    options.timeout = TIMEOUT
19    return options
20  }
21
22  // 响应拦截器配置
23  const responseInterceptor = (response) => {
24    const { data, statusCode } = response
25    // 网络状态码判断
26    if (statusCode !== 200) {
27      showToast(`请求失败: ${statusCode}`, 'error')
28      return Promise.reject(response)
```

```
29   }
30   // 业务状态码判断 (根据后端规范调整)
31   const { code, message, result } = data
32   switch (code) {
33     case 200:
34       return result // 成功返回业务数据
35     case 401:
36       // Token 过期, 重新登录
37       showToast('登录已过期, 请重新登录', 'warning')
38       removeToken()
39       uni.navigateTo({ url: '/pages/login/index' })
40       return Promise.reject(data)
41     default:
42       showToast(message || '操作失败', 'error')
43       return Promise.reject(data)
44   }
45 }
46
47 // 错误处理
48 const errorHandler = (error) => {
49   showToast('网络异常, 请检查网络连接', 'error')
50   console.error('请求错误: ', error)
51   return Promise.reject(error)
52 }
53
54 // 封装请求方法
55 export const request = (options) => {
56   // 请求拦截
57   const reqOptions = requestInterceptor(options)
58   // 发起请求
59   return uni.request(reqOptions)
60     .then(responseInterceptor)
61     .catch(errorHandler)
62 }
63
64 // 封装常用请求方式
65 export const get = (url, params = {}, header = {}) => {
66   return request({
67     url,
68     method: 'GET',
69     data: params,
70     header
71   })
72 }
73
74 export const post = (url, data = {}, header = {}, contentType) => {
75   return request({
```

```

76     url,
77     method: 'POST',
78     data,
79     header,
80     contentType
81   })
82 }
83
84 export const put = (url, data = {}, header = {}) => {
85   return request({
86     url,
87     method: 'PUT'

```

2. 状态管理 (store/modules/userStore.js)

Code block

```

1  import { defineStore } from 'pinia'
2  import { getToken, setToken, removeToken } from '../utils/auth'
3  import { login, getUserInfo, logout } from '../api/user'
4
5  // 定义用户状态存储
6  export const useUserStore = defineStore('user', {
7    // 状态
8    state: () => ({
9      token: getToken() || '', // Token 从缓存获取
10     userInfo: {}, // 用户信息
11     isLogin: !!getToken() // 登录状态
12   }),
13   // 计算属性 (类似 Vuex 的 getter)
14   getters: {
15     // 获取用户名
16     userName: (state) => state.userInfo.nickname || '游客',
17     // 获取用户头像
18     userAvatar: (state) => state.userInfo.avatar || '/static/img/default-
avatar.png'
19   },
20   // 方法 (类似 Vuex 的 mutation + action)
21   actions: {
22     // 登录操作
23     async loginAction(data) {
24       try {
25         // 调用登录接口
26         const res = await login(data)
27         // 存储 Token
28         this.token = res.token
29         setToken(res.token)
30         // 更新登录状态

```



```

31     this.isLogin = true
32     // 获取用户信息
33     await this.getUserInfoAction()
34     return res
35   } catch (error) {
36     console.error('登录失败: ', error)
37     throw error
38   }
39 },
40 // 获取用户信息
41 async getUserInfoAction() {
42   try {
43     const res = await getUserInfo()
44     this.userInfo = res
45     return res
46   } catch (error) {
47     console.error('获取用户信息失败: ', error)
48     throw error
49   }
50 },
51 // 退出登录
52 async logoutAction() {
53   try {
54     // 调用退出接口 (后端需要时)
55     await logout()
56   } finally {
57     // 清除状态
58     this.token = ''
59     this.userInfo = {}
60     this.isLogin = false
61     // 清除缓存
62     removeToken()
63     // 跳转到登录页
64     uni.redirectTo({ url: '/pages/login/index' })
65   }
66 }
67 }
68 })

```

3. 通用组件 (components/layout/NavBar.vue)

Code block

```

1  <template>
2    <view class="nav-bar" :style="{ backgroundColor: bgColor }">
3

```

```
4      <view class="nav-left" @click="handleBack" v-if="showBack">
5        <image src="/static/img/back.png" mode="widthFix" class="back-icon">
6      </image>
7    </view>
8
9    <view class="nav-center" :style="{ color: titleColor }">
10      {{ title }}
11    </view>
12
13    <view class="nav-right">
14      <slot name="right"></slot>
15    </view>
16
17    <view class="nav-placeholder" :style="{ height: navHeight + 'px' }"></view>
18  </template>
19
20  <script setup>
21  import { ref, onMounted } from 'vue'
22
23  // 组件属性
24  const props = defineProps({
25    title: {
26      type: String,
27      default: '页面标题'
28    },
29    showBack: {
30      type: Boolean,
31      default: true
32    },
33    bgColor: {
34      type: String,
35      default: '#ffffff'
36    },
37    titleColor: {
38      type: String,
39      default: '#333333'
40    }
41  })
42
43  // 导航栏高度（适配不同设备）
44  const navHeight = ref(44)
45
46  // 生命周期：获取导航栏高度
47  onMounted(() => {
48    // 获取系统信息
49    uni.getSystemInfo({
```

```

50     success: (res) => {
51         // 适配小程序胶囊位置
52         if (res.platform === 'devtools' || res.platform === 'ios' ||
res.platform === 'android') {
53             const menuButtonInfo = uni getMenuButtonBoundingClientRect()
54             navHeight.value = menuButtonInfo.top + menuButtonInfo.height +
(menuButtonInfo.top - res.statusBarHeight)
55         }
56     }
57 })
58 })
59
60 // 返回事件
61 const handleBack = () => {
62     const pages = getCurrentPages()
63     if (pages.length > 1) {
64         uni.navigateBack()
65     } else {
66         uni.switchTab({ url: '/pages/index/index' })
67     }
68 }
69 </script>
70
71 <style scoped>
72 .nav-bar {
73     position: fixed;
74     top: 0;
75     left: 0;
76     right: 0;
77     display: flex;
78     align-items: center;
79     justify-content: center;
80     z-index: 999;
81     padding: 0 16px;
82     box-sizing: border-box;
83 }
84
85 .nav-left {

```

4. 页面示例 (pages/login/index.vue)

Code block

```

1  <template>
2    <view class="login-page">
3
4      <NavBar title="登录" showBack="false" />

```

```
5
6
7   <view class="login-form">
8     <view class="form-item">
9       <CuInput
10         placeholder="请输入手机号"
11         v-model="form.phone"
12         type="number"
13         maxlength="11"
14       />
15     </view>
16     <view class="form-item">
17       <CuInput
18         placeholder="请输入密码"
19         v-model="form.password"
20         type="password"
21       />
22     </view>
23
24
25     <CuButton
26       class="login-btn"
27       :disabled="!form.phone || !form.password"
28       @click="handleLogin"
29     >
30       登录
31     </CuButton>
32
33
34     <view class="login-other">
35       <text @click="handleForgetPwd">忘记密码?</text>
36       <text @click="handleRegister">注册账号</text>
37     </view>
38   </view>
39 </view>
40 </template>
41
42 <script setup>
43 import { ref } from 'vue'
44 import { useUserStore } from '../store/modules/userStore'
45 import { showToast } from '../utils/common'
46
47 // 引入状态管理
48 const userStore = useUserStore()
49
50 // 表单数据
51 const form = ref({
```

```

52     phone: '',
53     password: ''
54   })
55
56   // 登录操作
57   const handleLogin = async () => {
58     // 简单验证
59     if (!/^1[3-9]\d{9}$/.test(form.value.phone)) {
60       return showToast('请输入正确的手机号', 'warning')
61     }
62     if (form.value.password.length < 6) {
63       return showToast('密码长度不能少于6位', 'warning')
64     }
65
66     try {
67       // 调用登录方法
68       await userStore.loginAction(form.value)
69       showToast('登录成功', 'success')
70       // 跳转到首页
71       uni.switchTab({ url: '/pages/index/index' })
72     } catch (error) {
73       console.error('登录失败: ', error)
74     }
75   }
76
77   // 忘记密码
78   const handleForgetPwd = () => {
79     uni.navigateTo({ url: '/pages/login/forgetPwd' })
80   }
81
82   // 注册账号
83   const handleRegister = () => {
84     uni.navigateTo({ url: '/pages/login/register' })
85   }
86 </script>
87
88 <style scoped>

```

五、模板使用说明

1. 快速初始化

1. 克隆/下载本模板到本地，修改 `manifest.json` 中的 `appid` 为自己的账号 AppID；
2. 执行 `npm install` 安装依赖包；
3. 在 HBuilderX 中打开项目，选择对应平台（微信小程序/App/H5）运行即可。

2. 扩展规范

- **页面扩展**：新增页面时在 `pages` 目录下创建对应文件夹，同时在 `pages.json` 中配置路由；
- **组件扩展**：基础组件放 `components/base`，业务组件放 `components/business`，全局组件在 `main.js` 中注册；
- **接口扩展**：新增接口模块在 `api` 目录下创建对应 JS 文件，统一使用 `request.js` 中的方法；
- **样式扩展**：主题色、字体大小等配置在 `theme.scss` 中定义，避免硬编码。

3. 多端适配注意事项

- **样式适配**：使用 `rpx` 作为尺寸单位，避免使用固定像素 `px`；
- **API 适配**：优先使用 Uniapp 内置 API，避免直接调用微信小程序或 App 原生 API；
- **权限适配**：不同平台权限申请方式不同，在 `manifest.json` 中按平台配置权限描述；
- **调试适配**：分别在对应平台的模拟器/真机中调试，确保功能正常。

六、优化建议

- **按需引入**：大型项目可使用分包加载，减少主包体积；
- **性能优化**：图片使用 CDN 存储，开启懒加载；页面使用 `onLoad` 初始化数据，避免重复请求；
- **安全优化**：敏感数据加密传输，Token 存储使用 `uni.setStorageSync` 并定期刷新；
- **日志监控**：集成第三方日志监控工具（如 Sentry），便于线上问题排查。

（注：文档部分内容可能由 AI 生成）